

CONFIDENTIAL ACADEMIC SUBMISSION - CSE499 review and grading only.

Mercury Project

Final Project Report

Project narrative, implementation summary, security hardening, testing, lessons learned, and future work.

Course	CSE499 Senior Project
Author	Matthew D. Barker
Version	RC 1.2 (1.2.0-rc.1)
Date	July 2026
Use	Academic review and grading only

This document and related project materials may not be publicly posted, redistributed, showcased, or shared outside the course review process without prior written permission from the copyright owner.

Document Purpose

This report summarizes the Mercury CSE499 senior project at the RC 1.2 release-candidate stage. It explains the problem, goals, architecture, implementation, security hardening, testing, demonstrations, limitations, lessons learned, and future work.

Executive Summary

Mercury is a modular secure communications framework for protected data exchange between software components. It protects payloads before transport, authenticates security-relevant SecureEnvelope fields, rejects replay attempts, and separates cryptographic logic from codecs, transports, replay protection, logging, and application hosts.

Mercury RC 1.2 is implemented as a C#/.NET solution with shared abstractions, a core client pipeline, AES-GCM and ChaCha20-Poly1305 providers, Binary and JSON codecs, in-memory, TCP, and FileDrop transports, console and WinForms demonstrations, and an xUnit automated test suite. The latest verification discovered 330 tests, passed 328, skipped the two pending AES-CCM tests, and reported zero failures.

Problem Addressed

Distributed systems often move data over channels that should not be trusted to provide confidentiality, integrity, authentication, or replay protection. When protection logic is mixed directly into application or transport code, it becomes difficult to test, difficult to extend, and easy to misuse.

Mercury addresses this problem by creating a reusable framework layer that protects data above the transport. The application provides a payload and context. Mercury protects it, transports only a protected frame, validates it on receipt, and releases plaintext only after all required checks succeed.

Project Goals

- Build a secure data exchange framework rather than a simple chat application.
- Demonstrate payload confidentiality, integrity, authenticated context, and tamper detection.
- Reject replayed protected frames.
- Show wrong-key and wrong-recipient failure behavior.
- Keep crypto, transport, encoding, replay, logging, and host concerns separated.
- Support multiple providers, codecs, transports, and application hosts through stable interfaces.
- Provide visible security demonstrations and extensive automated tests.
- Document architecture, threat assumptions, limitations, and acceptance evidence clearly.

Implementation Summary

Area	RC 1.2 Implementation
Language and platform	C# and .NET; framework contracts preserve their defined compatibility targets; demos and tests use current .NET targets.
Core API	IMercuryClient with SendAsync and ReceiveAsync, explicit ClientId, configured plugins, controlled results, and safe failure behavior.
Crypto providers	AesGcmCryptoProvider and ChaCha20CryptoProvider. AES-CCM remains pending.
Authenticated data	Length-prefixed canonical envelope data covering version, IDs, timestamp, algorithms, replay token, and header/footer metadata.
Envelope model	SecureEnvelope with Version, Header, protected Payload, and Footer.

Area	RC 1.2 Implementation
Codecs	Binary and JSON IEnvelopeCodec implementations.
Transports	In-memory, framed TCP, and FileDrop plugins.
Replay protection	Process-local InMemoryReplayProtector with expiration, concurrency safety, and bounded capacity.
Key handling	ISymmetricKeyProvider abstraction, defensive in-memory dictionary provider, explicit KeyId validation, and algorithm-specific key-size checks.
Demos	Cross-platform console host and Windows WinForms graphical demonstration.
Tests	xUnit unit, provider contract, codec, replay, transport, pipeline, failure, easy-setup, and integration coverage.

Architecture Summary

The architecture follows separation of responsibility. The application supplies payloads. IMercuryClient coordinates the pipeline. ICryptoProvider protects and authenticates payloads and canonical envelope data. IEnvelopeCodec serializes SecureEnvelope values. ITransport moves complete frames. IReplayProtector rejects duplicate authenticated communications. IMercuryLogger provides host-visible activity without coupling Mercury Core to a logging framework.

Application -> IMercuryClient -> ICryptoProvider -> SecureEnvelope -> IEnvelopeCodec -> ITransport -> Receiving IMercuryClient -> Application

The transport never receives plaintext. The codec never decides whether data is authentic. Replay state is updated only after provider authentication succeeds. Plaintext is returned only after decode, recipient, authentication, and replay checks succeed.

Security Summary

Property	How Mercury Demonstrates It
Confidentiality	AES-GCM or ChaCha20-Poly1305 encrypts payload data before transport.
Payload integrity	Changes to nonce, tag, ciphertext, or protected payload cause authentication failure.
Envelope integrity	Canonical authenticated data binds version, envelope ID, timestamp, sender, recipient, algorithms, replay token, and metadata.
Wrong-key rejection	Incorrect symmetric key material fails authenticated decryption and releases no plaintext.
Recipient validation	Explicit client KeyId and authenticated recipient context prevent unintended recipient acceptance.
Replay protection	Accepted sender/token combinations are rejected on duplicate delivery; storage expires and is bounded.
Safe failure	Decode, provider, authentication, replay, and transport failures do not release plaintext.
Transport independence	The same security pipeline operates above in-memory, TCP, and FileDrop transports.
Resource controls	TCP and in-memory frames are bounded, and replay storage has expiration and capacity limits.

Security Hardening Completed for RC 1.2

- Replaced limited delimiter-based authenticated context with deterministic length-prefixed canonical envelope data.
- Authenticated the complete header and footer metadata collections using ordinal key ordering.

- Added tamper tests for envelope ID, timestamp, signature algorithm, header metadata, and footer metadata across providers and codecs.
- Hardened KeyId and AlgorithmId default, empty, equality, and hash behavior.
- Hardened the in-memory symmetric key dictionary against empty identifiers, missing keys, null or empty key material, and mutable array exposure.
- Confirmed provider-specific 32-byte key validation for completed providers.
- Bounded in-memory replay storage and added expiration, concurrency, capacity, and recovery tests.
- Added maximum frame-size controls to the primary in-memory and TCP transport paths.
- Strengthened explicit client identity and recipient validation.
- Expanded failure-path testing to verify that plaintext remains empty after decode, authentication, replay, provider, and transport failures.
- Verified the WinForms attack simulator demonstrates replay and protected-payload tampering without placing attack logic in Mercury Core or the TCP transport.

Demonstration Applications

Console Demo

Mercury.Demo.Console provides a cross-platform menu-driven host. It demonstrates protected payload exchange, provider and codec selection, transport scenarios, and chunked file transfer without introducing a platform-specific UI dependency into the framework.

WinForms Demo

Mercury.Demo.WinForms provides the primary graphical demonstration. It includes configuration controls, payload and file sending, recovered result display, event logging, a protected SecureEnvelope viewer, status indicators, and controlled replay, tamper, and wrong-key scenarios. The demo uses custom Mercury controls and keeps attack simulation isolated from the production framework projects.

Testing Summary

The final suite is substantially broader than the early project plan. Tests now cover primitives, metadata, key providers, both completed crypto providers, both codecs, replay protection, in-memory, TCP, and FileDrop transports, the Mercury client pipeline, failure behavior, explicit recipient identity, authenticated envelope tampering, cancellation, large and binary payloads, easy setup, and multiple provider/codec combinations.

Release Evidence	Result
Automated tests discovered	330
Passed	328
Skipped	2 pending AES-CCM tests
Failed	0
Completed providers	AES-GCM and ChaCha20-Poly1305
Completed codecs	Binary and JSON
Verified demos	WinForms and console
Verified security scenarios	Tamper, replay, wrong key, malformed frame, recipient mismatch, and no-plaintext failure behavior

Major Design Decisions

Decision	Reason
Use a framework architecture instead of a single application	Demonstrates reusable senior-level design, clear contracts, and independent testing.
Separate crypto from transport and codec	Allows providers, encodings, and delivery mechanisms to change independently.
Use authenticated-encryption providers	Provides confidentiality and integrity without inventing custom cryptography.
Authenticate canonical envelope data	Protects security-relevant context and metadata from undetected modification.
Use explicit client identity	Makes recipient validation part of the core pipeline.
Apply replay protection after authentication	Prevents unauthenticated input from consuming trusted replay state.
Bound replay storage and frame sizes	Reduces avoidable resource-exhaustion risk.
Provide Binary and JSON codecs	Supports compact framing and readable diagnostics through the same envelope contract.
Provide console and WinForms hosts	Demonstrates portability and a strong evaluator-facing UI without coupling the framework to either host.
Document limitations and avoid certification claims	Keeps the submission technically honest and academically defensible.

Challenges and Lessons Learned

- Security properties must be stated precisely. Encryption alone does not prove replay resistance, intended-recipient validation, metadata integrity, or production readiness.
- A modular framework requires disciplined ownership of interfaces and project boundaries. Broad changes outside an assigned plugin create integration and review risk.
- Cross-platform requirements can change the best demonstration host. A console application provided portability without delaying the Windows graphical demo.
- Authenticated data must use deterministic encoding. Delimiter-based concatenation is weaker and harder to reason about than explicit length-prefixed canonicalization.
- Replay protection is a state and resource-management problem as well as a duplicate-detection problem.
- Tests are part of the security design. Contract and pipeline tests exposed regressions and proved that hardening applied consistently across providers and codecs.
- Large rewrites of stable code create unnecessary risk. Focused changes with immediate full-suite verification are safer.
- A demo must show the difference between structural decode failure and cryptographic authentication failure without placing attack behavior inside production components.
- Documentation must evolve with the code. Early drafts substantially understated the final providers, tests, authenticated data, replay controls, and demos.

Known Limitations

- No public key infrastructure, certificate-based identity, or human identity proofing.
- No production key exchange, long-term rotation, revocation, or hardware-backed key storage.
- No formal cryptographic proof or comprehensive side-channel hardening.
- Replay protection is process-local and resets when the application restarts.
- No production-scale rate limiting, distributed denial-of-service resistance, fuzz certification, or long-duration stress certification.
- AES-CCM remains pending and is represented by two intentionally skipped tests.
- FileDrop is included as a plugin but retains more recovery and production-hardening work than the primary in-memory and TCP paths.

- The graphical and console applications are demonstrations, not production operations consoles.
- Mercury RC 1.2 is an academic release candidate and is not presented as production-certified software.

Future Work

- Complete AES-CCM with provider contract, pipeline, codec, demo, and failure-path tests.
- Add digital signatures and certificate-backed identity where required.
- Add public key exchange, rotation, revocation, and secure key-storage integrations.
- Provide persistent or distributed replay protection.
- Complete additional FileDrop recovery, stale-claim, cleanup, and resource controls.
- Add more malformed-input fuzz testing, benchmarks, and long-duration stress testing.
- Add deployment-specific rate limiting, quotas, backpressure, and monitoring.
- Evaluate key-memory zeroing, disposal, HSM integration, and stronger deployment isolation.
- Add additional transport plugins without changing Mercury Core.
- Produce production deployment guidance only after a separate production-hardening effort.

Conclusion

Mercury RC 1.2 meets the academic goal of demonstrating a secure communications framework with modular architecture, authenticated encryption, authenticated envelope context, transport abstraction, codec abstraction, replay protection, bounded resource controls, tamper detection, extensive automated tests, and working console and WinForms demonstrations.

The project demonstrates more than CRUD functionality. It shows applied secure software engineering through clear boundaries, defensive programming, explicit threat assumptions, controlled failure behavior, reproducible testing, and documented design decisions.

Final mission statement: Protect the payload. Treat transport as untrusted. Fail safely. Prove it with code and tests.